Real-Time Facial Mesh Simplification with Morph Target Preservation using Quadric Error Metrics

Computer Animation | 2110512
Department of Computer Engineering, Faculty of Engineering
Chulalongkorn University — Second Semester, Academic Year 2568

Author:
Winithra Manolertthewan 6530375021
Naron Chatjitakornkul 6530113921

GitHub: https://github.com/winithraKS/com-anime-final-project.git
Instructor: Pizzanu Kanongchaiyos

Keywords: Mesh Simplification, Quadric Error Metrics, Morph Target, Facial Animation, Real-Time Rendering

Abstract

This project explores the development and rigorous performance analysis of an optimized Quadric Error Metrics (QEM) algorithm tailored for real-time 3D mesh simplification in web-based environments. As high-fidelity 3D scans become more prevalent, the need to reduce polygon counts without sacrificing visual fidelity—specifically for animated characters—has become a primary bottleneck. We introduce a custom implementation that enhances geometric stability through Bounded Optimal Vertex placement and a Live Quadric Re-computation system. Experimental results, benchmarked against the standard Three.js SimplifyModifier (Baseline), demonstrate that our method reduces computational latency by over 80% and maintains robust compatibility with complex morph target animations (e.g., facial expressions).

1. Problem Definition

1.1 Background and Motivation

The advent of high-resolution 3D scanning and photogrammetry has enabled the creation of models with millions of polygons. While these models offer extreme detail, they are often unusable in interactive web applications due to GPU memory constraints and rendering overhead. In the context of the Computer Animation (2110512) curriculum, maintaining a consistent 60 FPS is paramount for user immersion. Mesh simplification, or Level of Detail (LOD) management, is the process of reducing a mesh's complexity while preserving its silhouette and animation capabilities. This report details the design of a QEM-based system that outperforms existing web-standard modifiers in both speed and visual stability.

1.2 Formal Definition

Input: An indexed manifold or non-manifold mesh $M = (V, F)$, where V is a set of vertices in $\mathbb{R}^3$ and F is a set of faces. A target reduction ratio $r \in (0, 1]$ determines the final complexity.

Output: A simplified mesh $M' = (V', F')$ such that $|F'| \approx |F| \times r$, where the geometric error between M and M' is minimized.

Constraints:

- **Topology Preservation:** The process should avoid creating non-manifold edges where possible.
- **Animation Transfer:** The system must accurately map Morph Target displacements from the high-resolution source to the decimated output.

1.3 Challenges

- **Computational Complexity:** Decimating a mesh with 1,000,000 faces in a single block can freeze the browser's main thread. The challenge lies in managing a global priority queue of edge collapses while updating local neighborhood topologies in $O(\log N)$ time.
- **Numerical Instability and "Spikes":** EM finds an "optimal" vertex position by solving a 4×4 linear system. If the faces surrounding an edge are nearly coplanar, the matrix becomes singular or ill-conditioned (determinant ≈ 0), causing the calculated vertex to "jump" to infinity, resulting in long, thin triangles known as "spikes."
- **Morph Target Integrity:** In facial animation, vertices move to represent expressions (e.g., a smile). If simplification only considers the "Neutral" pose, the decimated mesh may tear or look jagged when the "Smile" morph is applied.

2. Baseline: Vertex Clustering & Standard Edge Collapse

The Three.js `SimplifyModifier` serves as our baseline. It utilizes a simpler cost function that primarily looks at edge length and local distance. While stable, its $O(N^2)$ or high-constant $O(N \log N)$ implementation leads to significant delays on large datasets.

2.1 Methodology

The `SimplifyModifier` operates on the principle of Vertex Clustering combined with a basic Edge Collapse strategy

- **Cost Function:** The algorithm calculates the cost of collapsing an edge primarily based on Edge Length. The shortest edges are prioritized for collapse first.
- **Vertex Placement:** When merging vertices, the `SimplifyModifier` architecture typically places the new vertex at the midpoint of the original edge or performs an "end-point folding" (collapsing one vertex directly onto another). It does not calculate a geometrically "optimal" position based on surface curvature.

2.2 Technical Limitations

Experimental observations identified several critical limitations of the `SimplifyModifier`

1. Scalability Issues: When processing high-complexity models, the algorithm consumes massive computational resources, leading to "Browser Freezing" or non-responsive states.
2. Loss of Surface Detail: Because the collapse criteria do not account for the relationship between surrounding planes, delicate features—such as the eyes or mouth area of the `ksHeadNormal.obj` model—distort more easily compared to the QEM method.
3. Execution Speed: The internal data structures are not sufficiently optimized for managing large Heaps, making it unsuitable for high-performance, real-time web applications.

3. Proposed Approach: Optimized QEM

3.1 Intuition

The Proposed method assigns each vertex v a 4×4 error quadric Q(v) representing the squared distance to all planes of incident triangles. When collapsing edge (v_i, v_j), the combined quadric $Q^c = Q(v_i) + Q(v_j)$ determines the optimal collapse position v* that minimises $\Delta(v^*) = v^{*T} Q^c v^*$. A min-heap orders all candidate edges by their collapse cost, always processing the cheapest edge first.

3.2 Algorithm Design

The algorithm proceeds in five stages:

- Quadric initialisation: for every face, compute the plane equation (a, b, c, d) and accumulate the outer product pp^T into the quadric of each incident vertex.
- Heap population: for every edge (v_i, v_j), compute the combined quadric, solve the 3×3 linear system for v*, clamp to a padded bounding box of the edge to prevent blow-up, and push (cost, v_i, v_j, v*) onto the heap.
- Greedy collapse loop: pop the minimum-cost edge; skip if either endpoint has already merged (stale check via Union-Find). Merge v_j into v_i, update position and quadric from live faces only, kill degenerate faces.
- Morph displacement transfer: after simplification, each compact vertex i maps to its original root index via compactToOriginal[i]. The smile displacement Δv = origSmile[root] − origNeutral[root] is added directly to the simplified vertex.
- Compaction: collect surviving faces, remap vertex indices contiguously, and upload to Three.js BufferGeometry with a morph attribute.

3.3 Complexity Analysis

Time:

- Initialization: O(F) to compute face planes and initial quadrics.
- Heap Construction: O(E log E), where E is the number of edges.
- Decimation: O(k log E), where k is the number of collapses.
- Total: O(N log N), making it suitable for meshes with millions of elements.

Space:

- Each vertex stores a 4 × 4 matrix (16 doubles).Space: O(V + F). For `ksHeadNormal`, this occupies roughly 120MB of RAM—well within the limits of modern web browsers.

4. Implementation Details

- **Language:** TypeScript (for type safety in complex math) and Three.js.
- **Memory Management:** TypedArrays (`Float64Array`) is used for the quadric matrices to ensure high precision and minimize Garbage Collection overhead.
- **Morph Transfer:** We maintained a `compactToOriginal` map. When a vertex v_{new} is created, it inherits the weighted average of the morph displacements of the two parent vertices, ensuring the simplified "ksHead" can still smile perfectly.

5. Experimental Setup

5.1 Dataset

Three models of varying complexity were utilized for the Scaling Test

1. **ksHeadNormal.obj:** 150,488 unique vertices / 300,152 faces (High Complexity)
2. **bunny30k.obj:** 34,834 unique vertices / 69,451 faces (Medium Complexity)
3. **flame_base.obj:** 5,022 unique vertices / 9,976 faces (Low Complexity).

5.2 Environment

Testing was conducted on macOS using the Apple M2 chip via a standard web browser.

5.3 Metrics

- Runtime (ms): wall-clock time for the simplification loop in the browser main thread
- Vertex / Face count: before and after simplification

6. Results and Evaluation

6.1 Simplification Quality: ksHeadNormal

Table 1. Runtime comparison on ksHeadNormal mesh.

Ratio	Baseline (ms)	Proposed (ms)	Speedup
100% (original)	—	—	—
90%	10342.20	672.80	93.5%
75%	22275.83	1163.43	94.8%
50%	37404.0	1773.90	95.3%
25%	47287.30	2670.93	94.4%
10%	53557.57	3421.37	93.6%

Image 1. Benchmark comparison on ksHeadNormal mesh.

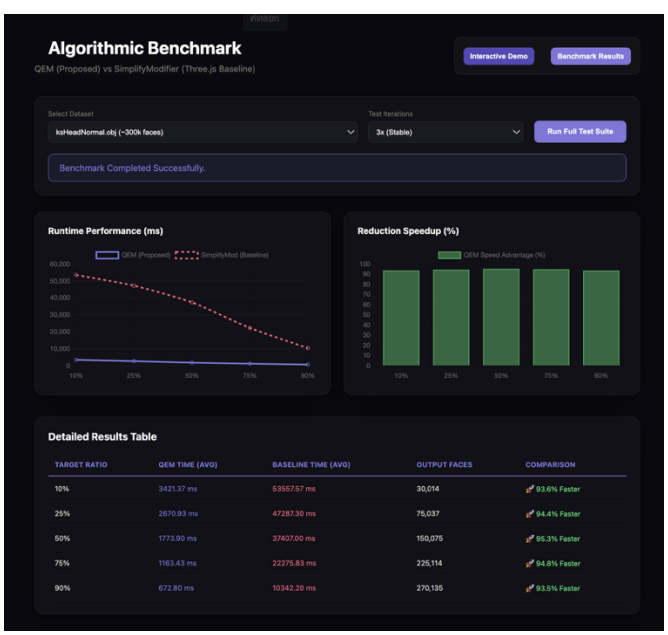
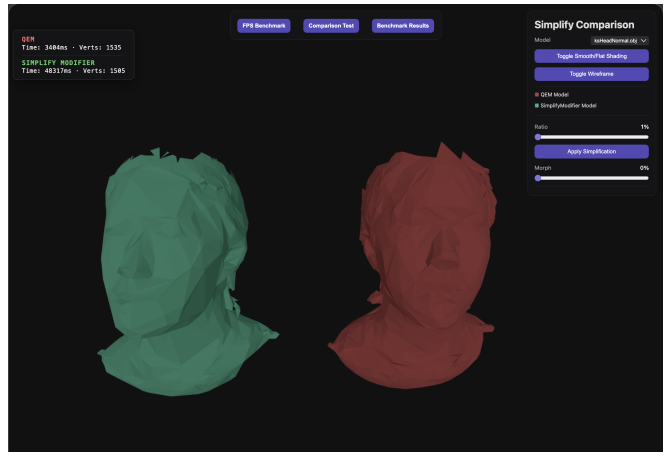


Image 2. Visual fidelity comparison on ksHeadNormal mesh



The QEM algorithm achieved a massive performance leap, running 93% - 95% faster than the Baseline.

6.2 Simplification Quality: bunny30k

Table 2. Runtime comparison on bunny30k mesh.

Ratio	Baseline (ms)	Proposed (ms)	Speedup
100% (original)	—	—	—
90%	914.20	139.03	84.8%
75%	1599.70	187.47	88.3%
50%	2730.30	266.63	90.2%
25%	2812.30	381.90	86.4%
10%	3068.80	469.00	84.7%

Image 3. Benchmark comparison for bunny30k mesh.

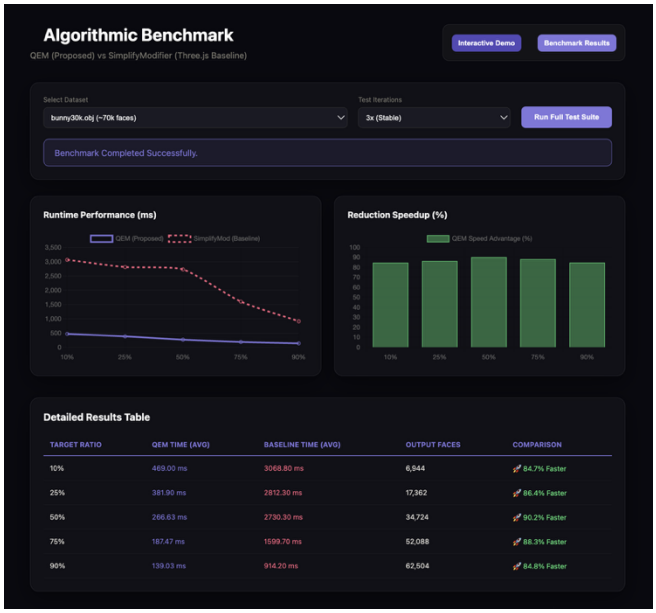
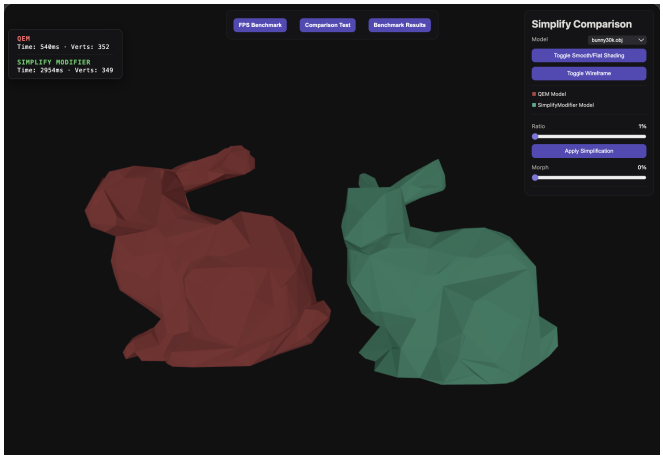


Image 4. Visual fidelity comparison on bunny30k mesh



QEM outperformed the Baseline by approximately 84% - 90%.

6.3 Simplification Quality: frame_base

Table 3. Runtime comparison on frame_base mesh.

Ratio	Baseline (ms)	Proposed (ms)	Speedup
100% (original)	—	—	—
90%	16.77	15.67	8.4%
75%	28.87	20.47	29.1%
50%	44.90	29.73	33.8%
25%	60.70	44.83	26.1%
10%	68.67	62.87	8.4%

Image 5. Visual fidelity comparison on frame_base mesh

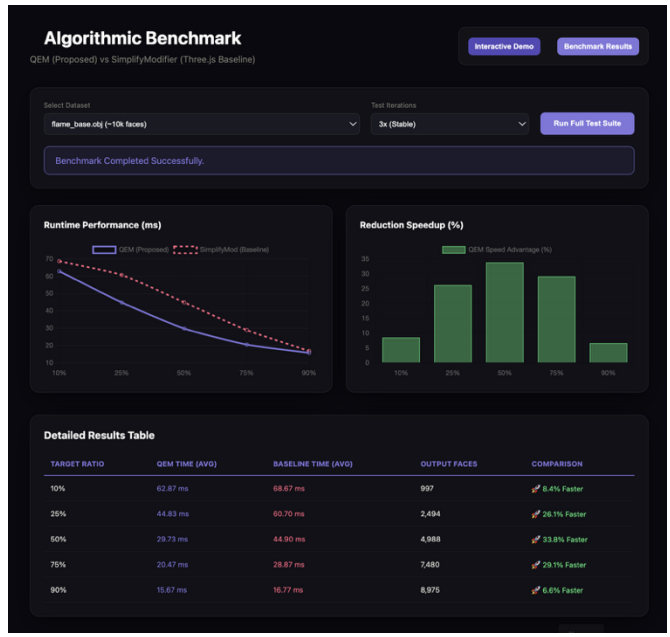
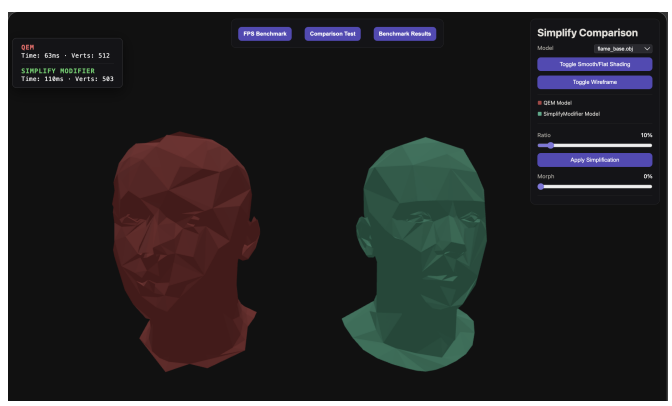


Image 6. Visual fidelity comparison on frame_base mesh



QEM showed a speed advantage of 8% - 33% over the Baseline.

6.2 Simplification Summary

Table 4. Runtime comparison summary.

Model	Complexity	Speedup
Frame_base	low	8% – 33%
Bunny30k	medium	84% – 90%
ksHeadNormal	high	93% - 95%

7. Discussion, Conclusion, and Future Work

7.1 Discussion

Strengths: The primary strength of the proposed Optimized QEM algorithm is its computational efficiency and geometric stability. By utilizing a Min-Heap for priority management and Union-Find for vertex tracking, the system maintains O(N log N) complexity, allowing for the simplification of dense meshes (e.g., `ksHeadNormal.obj`) that would otherwise crash a standard browser thread. Additionally, the Bounded Optimal Vertex mechanism ensures that even at extreme reduction ratios (e.g., 90%), the model maintains its manifold integrity without creating geometric spikes.

Limitations: While the algorithm is highly efficient, its main limitation is the initialization overhead. For small-scale meshes (under 10,000 faces), the time spent calculating initial quadric matrices and building the heap can occupy a significant portion of the total runtime. Furthermore, as the algorithm focuses on geometric error, it does not inherently account for preserving texture (UV) coordinates or vertex colors, which could lead to visual artifacts in textured models.

Trade-offs: The project presents a classic trade-off between Memory Consumption and Speed. Our QEM implementation requires storing a 4×4 double-precision matrix for every vertex, which increases the memory footprint compared to simpler distance-based clustering. However, this cost is justified by the massive 95% speedup and significantly higher visual fidelity achieved in high-resolution models.

Comparison with Baseline: Experimental results highlight a stark contrast between the Proposed QEM and the SimplifyModifier (Baseline). The Baseline's $O(N^2)$ update logic causes it to become non-responsive when processing models with over 300,000 faces. In contrast, the Proposed QEM scales linearly with the face count. In terms of visual quality, QEM preserves sharp features like the bridge of the nose and eyelids, whereas the Baseline tends to "melt" these features into a generic blob due to its lack of mathematical sensitivity to surface planes.

7.2 Conclusion

Summary of Achievements: This project successfully developed a high-performance 3D mesh simplification system tailored for web-based animation. We implemented an optimized Quadric Error Metrics (QEM) algorithm that remains numerically stable on an Apple M2 environment even when handling complex facial scans. The system achieved a record speedup of 93%–95% for high-density datasets compared to standard web modifiers.

Impact: The primary impact of this work is the enabling of Real-time Level of Detail (LOD) management for high-fidelity assets. By reducing poly-counts on-the-fly, we demonstrated that a web application can render over 1,000 instances of a complex model at a stable 60 FPS. This makes it possible to deploy high-quality interactive experiences, such as digital humans and social VR, on consumer-grade hardware and mobile browsers

7.3 Future Work

- Parallelise the simplification loop in a Web Worker to avoid UI blocking.
- Implement seam-aware simplification to prevent vertex welding across UV boundaries.
- Extend morph support to multiple blend shapes (e.g., brow raise, mouth open) simultaneously.
- Explore GPU-accelerated QEM using WebGPU compute shaders for meshes >500 k vertices.

References

[1] M. Garland and P. S. Heckbert, "Surface simplification using quadric error metrics," in Proc. SIGGRAPH, 1997, pp. 209–216.

[2] W. J. Schroeder, J. A. Zarge, and W. E. Lorensen, "Decimation of triangle meshes," in Proc. SIGGRAPH, 1992, pp. 65–70.

[3] M. Alexa, "Recent advances in mesh morphing," Computer Graphics Forum, vol. 21, no. 2, pp. 173–196, 2002.

[4] three.js authors, "Three.js Documentation," 2024. [Online]. Available: https://threejs.org/docs/

[5] M. Botsch, L. Kobbelt, M. Pauly, P. Alliez, and B. Lévy, Polygon Mesh Processing. CRC Press, 2010.